Understand the project. Explain the decisions. Defend the limits.

Eight-page study guide • Backend/engine version 0.2.0, with current frontend additions • Prepared 9 October 2026

1. Start with the business problem

A CRM (customer relationship management system) stores customers, contacts, companies, and deals. When records move between systems, IDs, field names, values, and links can change. Matching row counts alone does not establish that the right information moved.

This auditor checks **exported source and target CSV snapshots** against an explicit JSON contract. It identifies missing or unexpected records, changed mapped values, ambiguous identities, and declared parent-link problems. It produces reviewable evidence rather than a blanket migration certificate.

> **A 60-second explanation**
> “CRM Migration Auditor is a local full-stack application for checking migration exports before cutover. A reviewer creates a project, uploads immutable source and target snapshots, and declares field and relationship rules. FastAPI validates and stores the run; a shared execution service invokes a deterministic Python engine. React shows job state, comparison verdict, findings, input hashes, and downloadable reports. The design favors exact identity and explicit uncertainty over guessing. It is a synthetic-data portfolio application, not a production multi-tenant service.”

2. Learn the five core concepts

- **Project:** groups related exports and runs; not an ownership or tenant boundary.
- **Dataset:** one immutable CSV snapshot with a source/target side, exact headers, row count, and SHA-256 hash.
- **Contract:** declared ID columns, field transformations, comparisons, ID translations, and optional relationships.
- **Audit run:** a stored contract plus dataset references and execution state.
- **Report:** verdict, counts, bounded findings, declared scope, and provenance.

3. Study in this order

Run and inspect the demo (page 2). Trace the construction and architecture (page 3). Explain the comparison rules (page 4), backend consistency (page 5), and frontend/delivery choices (page 6). Practice technical answers and scenarios (pages 7–8).

Interview honesty: adapt the explanation to work you personally understand and contributed. Describe assistance, including AI tools, accurately if asked. Do not claim live CRM integrations, production users, exactly-once jobs, or measurements this project does not have.

Run it, then explain the result

Use Python 3.14 and Node.js 24. From the repository root, the Windows setup is:

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.lock
.\.venv\Scripts\python.exe -m pip install --no-deps -e .
.\.venv\Scripts\python.exe -m uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

On macOS/Linux, create the environment with `python3.14 -m venv .venv` and replace the executable path with `.venv/bin/python`. Use the same interpreter for installation, runtime, and checks.

Keep the API running. In a second terminal:

```
cd frontend
npm ci
npm run dev
```

Open `http://127.0.0.1:5173` and select **Run synthetic demo**. No Docker, Redis, cloud credentials, or AI key is needed. SQLite and uploaded snapshots live under `.data/`.

What to demonstrate

1. Show the stored contract: `Id` matches `LegacyId`; email uses `strip` then `casefold`; stages use explicit translations; amounts use Decimal tolerance `0.01`.
2. Explain why execution is **completed** while comparison is **failed**. The software ran successfully and found data discrepancies.
3. Filter findings, inspect exact JSON, and point to input hashes and scope.
4. Download JSON/CSV, then reopen the run from audit history. Filters affect the view, not the exported report.

Evidence	Expected bundled-demo result
Coverage	3 source rows, 3 target rows; 2 matched records; 6 field comparisons.
Changed field	`field_mismatch`: record `002`, amount `200` versus `250`.
Missing / unexpected	Source record `003` has no target match; target record `004` has no source origin.

Read next: compare `examples/source.csv`, `target.csv`, and `contract.json` with the UI. To recreate the run without React, use `python scripts/demo.py --export-dir .data/demo-exports` with the project interpreter. Swagger is at `http://127.0.0.1:8000/docs`.

Reference: `README.md`, `examples/`, `scripts/demo.py`, `frontend/src/components/SyntheticDemo.jsx`. The bundled demo covers fields, not parent relationships.

How the application is built

The table maps the current implementation. The steps below are a dependency-aware way to reconstruct it, not a claim about the historical order of commits.

Layer / technology	Responsibility and code to inspect
Domain / Python + Pydantic	`app/domain/`: typed contracts, CSV parsing, pure comparison, findings/report schemas. No HTTP or database dependency in the engine.
Persistence / SQLAlchemy	`app/db/`: projects, datasets, audit runs, sessions. SQLite locally; optional PostgreSQL. Alembic in `migrations/` versions schema changes.
Use case / services	`app/services/`: immutable file storage, checksum verification, job claiming, engine invocation, result persistence.
Transport / FastAPI	`app/api/`: request validation, project scoping, uploads, dispatch, status, retries, and report downloads. Uvicorn serves the ASGI application.
Execution / task adapters	Local FastAPI background task or `app/workers/celery_app.py`. Celery uses Redis as broker; the database, not a Celery result backend, holds run state.
Presentation / React + Vite	`frontend/src/`: project and audit screens, contract builder, polling, evidence review. Vite develops/builds the UI; nginx serves packaged assets.

Reconstruct the build in six steps

1. **Define the contract first.** Specify exact IDs/headers, allowed normalizers, comparisons, relationship rules, and invalid/ambiguous cases. Pydantic rejects unknown properties and invalid maps.
2. **Implement and exercise the pure engine.** Parse bounded inputs, index records, compare declared rules, and calculate verdicts. Domain tests make correctness independent of web infrastructure.
3. **Add persistence and snapshots.** Store metadata in three tables, original CSVs in UUID-named files, and contracts/reports as JSON. Local `create_all` bootstraps tables, not upgrades. Managed deployments disable auto-creation and apply Alembic migrations before startup.
4. **Expose an API and shared execution service.** Validate project/side/header references before creating runs. Commit a queued run, dispatch its UUID, claim it, verify bytes, compare, and persist completion or failure.
5. **Build the reviewer workflow.** React adds uploads, rule controls, history, status/error states, provenance, filters, and downloads. The guided demo uses the same real API.
6. **Package and verify the boundaries.** Lock dependencies, build static entries, add same-origin proxies, Compose services, domain/API tests, and a built-browser end-to-end regression.

Why these boundaries? Rules remain independently testable; local and Celery execution reuse one use case; the browser never decides the authoritative audit verdict.

Read in order: `contracts.py`, `csv_reader.py`, `engine.py`, `db/models.py`, `services/audits.py`, `api/routes.py`, then `frontend/src/App.jsx`.

Explain the engine precisely

Identity before values

`compare()` groups records by exact key strings. Leading zeros and case are preserved. An explicit one-to-one `key_map` translates source IDs; unmapped IDs use identity. Duplicate/blank keys and collisions between explicit mapping and identity are not guessed. Ambiguous records are excluded from conclusive comparisons.

Field processing

For each unique matched record, normalize the source value in declared order, apply its optional source-value map, and normalize the mapped result again. Normalize the target value with the same operations. Compare exact text, or evaluate `abs(expected - actual) <= tolerance` with Python Decimal.

Numeric cells use strict ASCII decimal syntax and bounds: at most 100 characters, 38 coefficient digits, and Decimal tuple exponent from −18 to +18. Blanks, underscores, non-finite numbers, and implicit whitespace are invalid; whitespace needs an explicit `strip`. Contract tolerance validation is separate: normalized precision is limited to 38 digits and 18 decimal places (at most 20 whole digits).

Verdict	Meaning and precedence
`not_verified`	Uncertainty exists: duplicate/empty keys, invalid values, ambiguous mappings, empty datasets, or missing parents. Takes precedence even if discrepancies also exist.
`failed`	No uncertainty finding, but a conclusive discrepancy exists: missing/unexpected records, field differences, or relationship mismatch.
`passed`	No findings within the declared snapshots/rules. Not proof of export completeness or live-system correctness.

Relationships: contact → company as an example

A named parent pair references uploaded source/target company snapshots and their ID columns. A child rule identifies each contact's company-reference columns. If source parent `A` explicitly maps to target parent `X`, the engine must establish unique parent identities and check that the migrated contact points to `X`.

If it points to another verified parent `Y`, that is `relationship_mismatch`. If a necessary parent is absent or ambiguous, verification is uncertain. Two exactly empty references pass only with `allow_empty: true`. Reference IDs are not normalized; company fields and transitive relationships are not automatically checked.

Determinism, limits, and complexity

Sorted keys and stable rule order make findings reproducible. The default report retains 1,000 findings, but counts and verdict cover all admitted records; evidence strings are bounded. Default admission is 5 MiB/CSV, 25,000 records/CSV, and 200 columns. Parsing and indexes are in memory.

Complexity reasoning, not a benchmark: with fixed limits and rules, indexing is approximately linear; sorting distinct keys adds log factors. Field/link work grows with matched records × declared rules. Dictionary access is average O(1); memory still includes complete parsed primary/parent tables, not just retained findings.

Reference: `app/domain/engine.py`, `contracts.py`, `csv_reader.py`, `tests/test_engine.py`, `tests/test_relationships.py`.

Trace requests and failure modes

Business endpoints are under `/api/v1`. Replace `{p}` and `{a}` with project/audit UUIDs. Lists use bounded `limit` and non-negative `offset`.

Request	Contract
`POST /projects`	Create a project: HTTP 201.
`POST /projects/{p}/datasets?side=source`	Upload multipart CSV; repeat with target side: HTTP 201. Invalid input: 422; oversize admitted file: 413.
`POST /projects/{p}/audits`	Dataset UUIDs plus contract; accepted run: 202. Invalid references/side/headers are rejected before execution.
`GET /projects/{p}/audits/{a}`	Execution state. Report endpoints add `/report`, `/report.json`, or `/report.csv`; unavailable before completion: 409.
`POST …/{a}/retry`	Only failed execution can requeue: 202, otherwise 409. Corrected data or rules need a new run.

Persisted facts and immutable evidence

`Project` stores name/description; `Dataset` stores project, side, filename, headers, row count, and hash; `AuditRun` stores dataset references, contract, state, timestamps, error code, and report. CSV bytes are outside the database. Storage uses UUID paths and exclusive creation; client filenames never become paths. Execution rechecks project/side and SHA-256 before parsing each distinct snapshot.

Lifecycle and concurrency

A run moves from queued to running, then completed or failed. A conditional SQL update claims only a queued run; only the successful claimant executes. This prevents concurrent duplicate delivery from starting the same queued run twice. It does **not** guarantee exactly-once processing or automatic recovery.

Creating the database row and publishing to the broker are separate operations. A crash between them can strand a queued run; process shutdown or worker failure can strand work. The current recovery for interrupted queued/running work is a new run, retaining old evidence. Reliable hosting would require a transactional outbox, expiring leases, and attempt fencing.

Terms to explain: an outbox stores the job and a publish intent in one database transaction, then a dispatcher publishes it. A lease is a time-limited worker claim. Fencing requires the current attempt token when persisting results, preventing a stale worker from overwriting a newer run attempt.

Security: what exists and what does not

An optional shared `X-API-Key` guards business endpoints; project-scoped checks prevent mixing references across projects. UUID secrecy is not user authorization. Public hosting needs authenticated users/tenants, ownership checks, quotas, pre-parser body limits, retention/deletion, encrypted storage, and backups.

Error logs avoid uploaded values and exception text. Reports deliberately contain evidence and require the same access checks. CSV export prefixes formula-like cells; JSON preserves exact report values. A SHA-256 digest establishes byte identity, not exporter authenticity or completeness.

Reference: `app/api/routes.py`, `report_exports.py`, `app/db/models.py`, `app/services/audits.py`, `docs/architecture.md`.

The user interface and release path

React behavior to understand

- **Routing:** hash routes select projects/audits. UUIDs are canonicalized to lowercase. The skip link focuses main content without changing the route.
- **Reads:** `useApiResource` uses AbortController cleanup and ignores aborted responses so older requests cannot overwrite new state.
- **Writes:** creation/upload/submission controls have independent in-flight locks and errors. A UI lock prevents accidental double submission; it is not server-wide idempotency.
- **Polling:** queued/running jobs schedule the next fetch after success, about every 1.5 seconds. Terminal states or request errors stop polling. Manual reload resumes it.
- **Evidence:** a failed execution refresh preserves a loaded completed report and its filters with a last-known-state warning. Report reload has its own error handling.
- **Drafts and scope:** builder state survives project-tab switches but not route departure. Finding filters/page controls cover retained evidence only; downloads cover the stored report.

How requests reach the backend

Vite proxies same-origin `/api`, `/health`, `/docs`, and `/openapi.json`. Optional `API_PROXY_KEY` stays in the Node process, not React or browser storage. A `VITE_` prefix would expose it. This shared-key delegation is not public authentication.

The build includes workspace/gallery entries; Vite preview is not a production server. Compose packages PostgreSQL, Redis, migrations, API, worker, and nginx-served React. Loopback ports are browser 8080 and API 8000. nginx injects no key: a keyed backend rejects business UI requests. Use Swagger authorization or the configured local Vite proxy.

Verification commands

```
# Repository root, using the project Python interpreter:
python -m pytest --cov=app --cov-report=term-missing
python -m ruff check .
python -m ruff format --check .
# frontend/ (after page 2 setup):
npm run build
npx playwright install chromium
npm run test:e2e
```

Playwright exercises built assets, a real API, and temporary SQLite storage: uploads/rules, findings, history, exports, precision boundaries, keyboard navigation, uppercase IDs, and offline refresh. CI adds PostgreSQL migration and Compose smoke jobs.

> **Evidence you can cite accurately**
> The portfolio-completion run observed 55 backend tests passing, 97% app coverage (excluding the Celery adapter), Ruff passing, a successful production build, and one passing Chromium end-to-end test. Desktop/mobile smoke was exercised. Docker was unavailable: no local container/PostgreSQL/Celery runtime success was observed.

Reference: `frontend/src/hooks/`, `components/AuditPage.jsx`, `vite.config.js`, `playwright.config.js`, `compose.yaml`, `.github/workflows/ci.yml`.

Practice concise technical answers

Answer with: decision → mechanism → tradeoff → evidence. These are outlines, not a script to memorize.

1. Why FastAPI and Pydantic?

Typed request/response models, validation, dependency injection, and generated OpenAPI support an inspectable contract-driven API. Pydantic validates rules; routes validate project membership, dataset side, and required headers. Neither replaces engine tests or per-user authorization.

2. Why a pure comparison engine?

It separates business decisions from HTTP, storage, and queues. The same deterministic function can be tested with tables and used by local or Celery execution. Infrastructure remains in the service layer, so database failures are not confused with comparison verdicts.

3. Why explicit mappings instead of fuzzy matching or AI?

Wrong identity matches can create false assurance. Exact legacy IDs and approved one-to-one crosswalks are explainable and reproducible. Fuzzy/AI mapping suggestions could be reviewed separately in future; they should not silently determine the final audit.

4. Why not compare amounts with float?

Binary floating-point can misrepresent decimal values near tolerance boundaries. Decimal plus strict input bounds and sufficient arithmetic precision supports exact subtraction for admitted inputs. The rule is absolute tolerance, not currency conversion or percentage tolerance.

5. Why SQLite locally and PostgreSQL optionally?

SQLite removes setup friction for the synthetic local demo. SQLAlchemy and Alembic provide a path to PostgreSQL, but portability is not proof of identical locking or operational behavior. Test migrations and workload/concurrency on the intended database before hosting.

6. Why store findings as JSON?

A bounded report naturally belongs to an immutable run and is portable across the supported databases. It simplifies the local implementation. For exhaustive server-side filtering, analytics, or per-finding reviewer decisions, use a findings table and appropriate indexes.

7. What do HTTP 202 and “completed” actually mean?

202 accepts a run, not its comparison result. Completed means execution produced a report; that report may pass, fail, or remain unverified. A failed execution has an error code and no usable comparison verdict. The UI exposes those concepts separately.

8. Is the background work asynchronous and durable?

It is deferred from the request. Local synchronous work runs through FastAPI’s background-task mechanism, not a durable job queue. Optional Celery decouples execution with a broker, but the current database/broker crash window and missing recovery leases remain.

9. What does the hash prove, and is the app secure?

The hash ties execution to stored input bytes; it cannot prove a complete or trustworthy export. UUID paths, bounds, scoped checks, safe exports, and a shared key are useful controls. They do not provide tenants, per-user authorization, retention, or production upload isolation.

10. What is the strongest testing evidence?

Business-edge tests plus a built-browser workflow using the real API and persisted reports. Explain a concrete boundary or state-transition regression rather than treating the coverage percentage as a correctness certificate. Name untested operational boundaries explicitly.

Prepare for “what happens if…?”

11. Both exports have the same number of rows. Why fail?

Counts ignore identity and values. The bundled example loses record 003, adds 004, and changes 002’s amount while keeping equal row counts. Show record-level findings instead of arguing from aggregate totals.

12. The source has duplicate IDs, or a mapping collides?

Do not select the first record or invent merge behavior. Duplicate/ambiguous identities produce uncertainty and are excluded from conclusive comparisons. Explicit maps are one-to-one; runtime also detects collisions with unmapped identity destinations.

13. Two workers receive the same audit, then one crashes?

The conditional queued-state claim allows one execution to start. A crash after claiming may leave running state indefinitely. Explain the limitation; propose leases and attempt tokens so an old worker cannot overwrite a newer attempt, plus an outbox for reliable dispatch.

14. The API is ready, but the audit never finishes?

Readiness checks database/schema, not workers. Inspect execution status, task-backend configuration, broker/worker availability, and shared snapshot storage. Inspect safe error codes/logs. Do not infer a passed verdict or blindly requeue running work.

15. The customer wants millions of rows or merged records?

Do not merely raise limits. Benchmark CPU/memory, account for parent tables, and design chunking or a disk-backed join/analytical engine with deterministic output and resource budgets. Merges need new explicit identity semantics; the current one-to-one model deliberately rejects them.

16. What real bug can you explain in detail?

Use the skip-link defect: `#main` was interpreted as a hash route, unmounting the active page. Reproduce with keyboard activation, prevent route navigation, explicitly focus main, and verify route/report-filter preservation in Chromium. If using this story, describe your contribution and assistance accurately.

A rehearsal you can repeat

1. **Pitch:** explain the customer problem and scope in one minute.
2. **Demo:** show findings, rules, hashes, history, and the CLI fallback.
3. **Code:** trace `create_audit → dispatch → execute_audit → compare`.
4. **Tradeoff:** defend a choice, name its limit, propose a concrete mitigation.
5. **Ownership:** distinguish your contributions, assistance, and independently verified work.

> **Ready-to-interview checklist**
> Can you predict a mismatch versus an uncertain verdict; configure a parent-link rule; explain a crash window; point to a regression; distinguish observed checks from configured CI; and name the first production-hardening priorities? If not, revisit the corresponding page and code before rehearsing again.

First hosting priorities: tenant authorization and pre-parser upload controls; durable dispatch/recovery; secure storage/retention/backups; workload limits and operational monitoring. Live connectors and AI suggestions are separate product features, not fixes for those boundaries.